Detecting Automated and Anomalous Editing Behaviour on Wikimedia at Scale

Mining of Massive Datasets (23CSE355) — End Semester Project

Spark MLlib on 11.0 million Wikimedia edits: bot classification and editor clustering

Team _No: [Team No] Project Advisor: [Advisor Name]

Sl. No.	Reg. No.	Name of the student
1	[Reg No 1]	[Name 1]
2	[Reg No 2]	[Name 2]
3	[Reg No 3]	[Name 3]
4	[Reg No 4]	[Name 4]

Introduction

Wikimedia projects accept edits from anyone, with no review before publication. Across all projects the platform absorbs roughly 30–50 edits per second — far beyond what volunteer reviewers can inspect. A large share of that traffic is automated: approved maintenance bots, semi-automated human tooling, and unapproved scripts.

This project asks two questions of the same corpus. First, **is this edit automated?** — a supervised classification problem, because reviewer attention should not be spent on predictable machine edits, while unflagged automation is a policy violation. Second, **what kinds of editors exist?** — an unsupervised clustering problem, because rate limits and review tooling should be designed around real behavioural populations rather than assumed ones.

We process the complete all-time edit history of Simple English Wikipedia — 15,201,370 dump rows containing 11,023,987 edits — in Apache Spark, engineer behavioural features, reduce comment text with SVD, and compare three classifiers against three clustering algorithms in MLlib.

Dataset Overview

Item	Details
Source	Wikimedia mediawiki_history dumps, snapshot 2026-10 (CC0)
Wiki / period	simplewiki, complete all-time history (2001–2026)
Download	951 MB bz2 TSV, 78 tab-separated columns
Raw rows	15,201,370 (revision 11,023,987, page 2,231,160, user 1,946,223)
Edits analysed	10,987,077 after cleaning (0.33% removed)
Attributes used	31 of 78 columns: actor, page, namespace, byte sizes, comment, cadence, revert links, tags
Labels	label_bot 23.7% positive; label_reverted 14.1% positive
Editors	780,867 distinct actors (registered accounts + anonymous IPs)
Live companion	stream.wikimedia.org EventStreams (SSE) for the demo

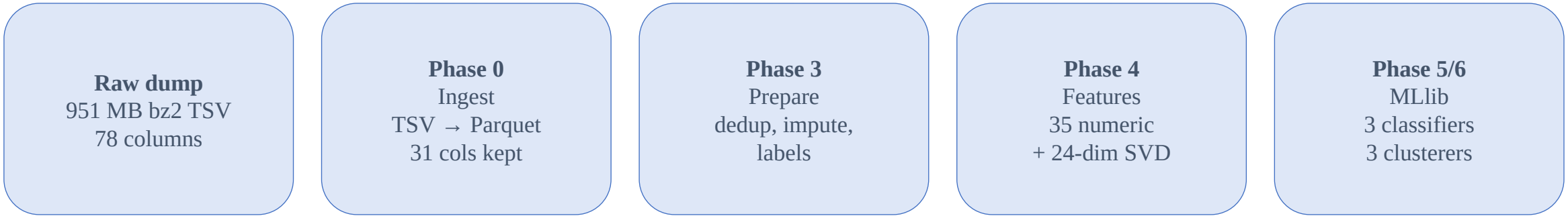
Problem Definition and Why This Dataset

A classification project is only meaningful if its label is genuinely learnable. We chose Wikimedia specifically because of label validity, and rejected GDELT 2.0 for failing that test.

	Wikimedia (chosen)	GDELT 2.0 (rejected)
Candidate label	event_user_is_bot_by	QuadClass / GoldsteinScale
Where it comes from	MediaWiki user-group membership and the bot-name registry — an administrative fact about an account	A deterministic lookup table keyed on the EventCode column
Is it learnable?	Yes — it is not a function of any feature we engineer	No — keep EventCode and the model reproduces a lookup table; drop it and the task is ill-posed

Interrogating label validity before modelling is the core methodological decision of this project.

Pipeline Architecture



Driver and executors run inside one JVM on a laptop (**local[*]**, 12 cores): the driver builds the DAG, Spark's local scheduler stands in for the cluster manager, and executors are threads. The same code runs on a real cluster unchanged.

Each phase persists Parquet, so any phase can be re-run in isolation without repeating the expensive bz2 decompression upstream.

Environment and Spark Configuration

```
src/config.py
SPARK_CONF = {
  "spark.driver.memory": "6g",
  "spark.sql.shuffle.partitions": "48",   # ~4x cores
  "spark.sql.adaptive.enabled": "true",
  "spark.sql.adaptive.skewJoin.enabled": "true",
  "spark.serializer": "...KryoSerializer",
}
MASTER = "local[*]"
```

Component	Version / value
Python	3.11.18 (via uv)
PySpark	3.5.3
Java	OpenJDK Temurin 17.0.21
Hardware	12 cores / 15 GB, local[*]
Shuffle partitions	48
Parquet written	1.1 GB (from 951 MB bz2)

Two version traps, both of which fail at runtime rather than at import:

PYSPARK_PYTHON must be pinned. System Python here is 3.14; if workers resolve bare python3 while the driver runs the venv, Spark dies mid-task with PYTHON_VERSION_MISMATCH.

PySpark 3.5 cannot run on Python 3.12 — pyspark/ml/image.py imports distutils, removed in 3.12. Support arrived in Spark 4.0, so we use 3.11.

Literature Survey

#	Work	Dataset	Algorithms	Metrics	What we take
1	Halfaker & Taraborelli, ORES (2015)	Wikipedia revisions	GBT, RF	precision/recall, AUC	reviewer routing needs precision
2	Potthast et al., ECIR 2008	940k en-wiki edits	logistic regression	P / R / F1	comment-shape features
3	Adler et al., CICLing 2011	PAN-WVC-10	RF, feature ablation	ROC-AUC, PR-AUC	structural-vs-full ablation
4	Davis et al., Botometer (2016)	Twitter accounts	random forest	ROC-AUC	cadence regularity as the signal
5	Chu et al., IEEE TDSC 2012	500k Twitter accounts	entropy + Bayesian	accuracy, FP rate	variance, not volume
6	Welser et al., iConference 2011	Wikipedia editors	descriptive clustering	role validity	editor-archetype framing
7	Yang et al., ICWSM 2016	Wikipedia histories	k-means, mixtures	silhouette	k by silhouette + naming
8	Kumar et al., VEWS, KDD 2015	33k Wikipedia users	SVM, RF	early detection	per-editor modelling
9–10	Zaharia et al. NSDI 2012; Meng et al. JMLR 2016	—	RDD model, MLlib	runtime scaling	Phase 7 infrastructure

Three threads: edit-level abuse detection (1–3, 8), automation by timing (4–5), unsupervised editor roles (6–7).

Phase 3: Data Preparation

Cleaning runs on Spark DataFrames: de-duplicate on the natural key, recover deleted-page rows, impute where 'missing' carries meaning, then derive the two labels.

src/p03_prepare.py

```
# deleted pages null the current-state columns but keep *_historical
df = (df.withColumn("page_namespace",
          F.coalesce("page_namespace", "page_namespace_historical"))
        .withColumn("page_title",
          F.coalesce("page_title", "page_title_historical")))

df = df.dropDuplicates(["revision_id"])

# NULL == "true" is NULL in SQL -> would become NaN downstream
for c in BOOL_COLS:
    df = df.withColumn(c,
        F.coalesce(F.col(c) == F.lit("true"), F.lit(False)))

df = (df.withColumn("label_bot",
          (F.length(F.trim("event_user_is_bot_by")) > 0).cast("int"))
        .withColumn("label_reverted",
          F.col("revision_is_identity_reverted").cast("int")))
```

Step	Rows
Revision rows in	11,023,987
Duplicates removed	0
Null-key rows removed	36,910
Clean rows	**10,987,077**
Bot edits	2,598,870 (23.7%)
Reverted edits	1,550,915 (14.1%)

Only 0.33% of rows were discarded, and a null guard asserts every feature-input column is null-free before writing.

AMRITA VISHWA VIDYAPEETHAM

Phase 4: Feature Engineering

35 numeric edit-level features plus a 24-dimension comment block, and 19 aggregated per-editor features for clustering.

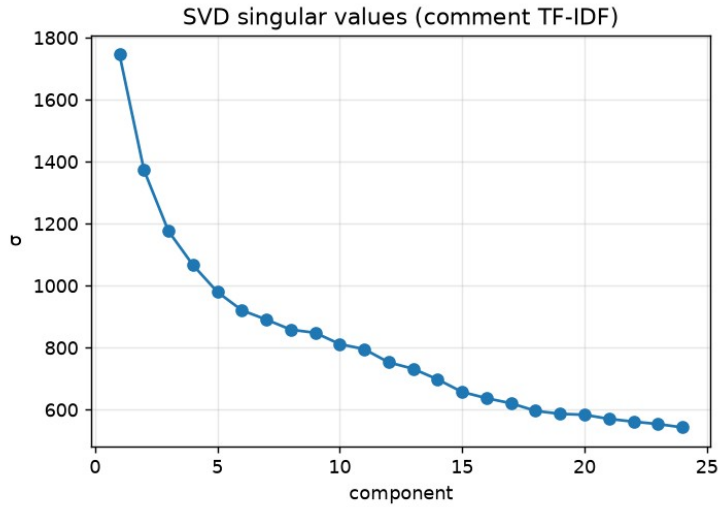
Group	Features	Intuition
Temporal	hour, day-of-week, weekend, night, hour sin/cos	bots run on cron and show a flat 24h profile; humans sleep
Edit size	bytes, signed diff, |diff|, log|diff|, diff ratio	maintenance bots make small, uniform edits
Cadence	log seconds since previous edit, has-previous, log edit count	machine timing is regular; human timing is bursty
Account	age in days, is-anonymous	separates IPs, new accounts and long-lived tools
Page context	namespace, is-content, is-redirect, is-minor, is-create	bots concentrate in File/Template namespaces
Comment shape	length, words, /* section */, wikilinks, URLs, digit / upper / punct ratio	structure of the message, not its vocabulary
Tags	tag count, mobile, visualeditor, undo	bot-naming tags excluded to avoid leakage
Editor-level	gap CV, night ratio, minor ratio, revert ratio, edits per active day, pages per edit	behavioural profile for clustering

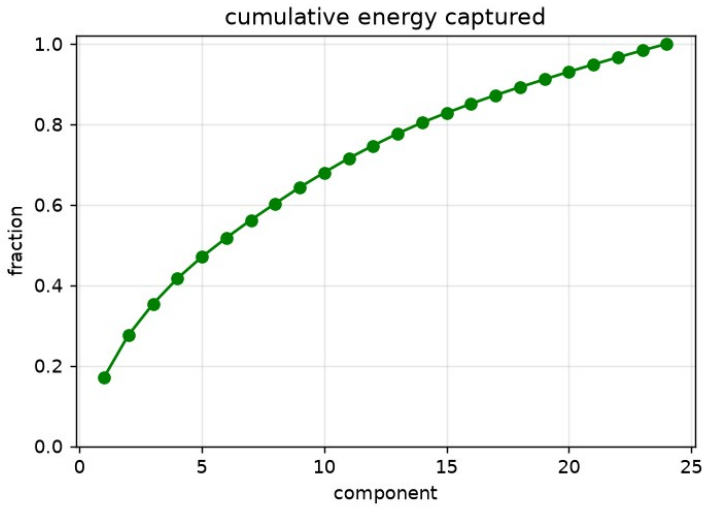
e_gap_cv — the coefficient of variation of inter-edit gaps — is the single most direct 'metronomic automation' signal.

Phase 4: Dimensionality Reduction with SVD

Comment text is tokenised, hashed into 2048 TF-IDF buckets, then reduced to 24 dimensions by a **truncated SVD** on a distributed mllib RowMatrix. Only the right singular matrix V (2048 x 24) is collected and broadcast; the projection X.V then runs per partition.

The SVD is **fitted on a 4% sample** — the right singular vectors converge with far fewer rows, which cut the fit from 21.5 minutes to a few — but the projection is applied to all 10,987,077 rows. Top singular values: 1746, 1373, 1176, 1067, 980, 921.





A genuine SVD rather than PCA: the decomposition is left uncentred so the sparse TF-IDF structure is preserved.

Leakage Control and the Actor-Disjoint Split

Three separate leaks had to be closed before any score could be trusted.

Leak	Why it leaks	Control
Username / user groups	usernames literally contain 'Bot' (e.g. SchlurcherBot); event_user_groups contains the bot group that DEFINES the label	excluded from every feature vector; username used only as a grouping key
Random row split	the same bot account would appear in train and test, letting the model memorise per-account cadence fingerprints	hash actor_key into disjoint buckets; the script asserts 0 shared actors
Scaler fitted on all data	StandardScaler means/variances would encode test-set statistics	scaler sits INSIDE the ML Pipeline, so it is fitted on the training fold only

Comment text is kept deliberately. It is observable at edit time and is not derived from group membership, so it is a legitimate predictor — but to keep that claim honest every model is trained twice, with and without the text block, so its contribution is measured rather than assumed.

Verified at run time: actor-disjoint hash split, 8,619,472 train / 2,367,605 test rows, 0 shared actors.

Phase 5: MLlib Implementation

src/p05_classification.py

```
def actor_split(df, frac, seed):
    b = F.abs(F.hash(F.concat(F.col("actor_key"),
              F.lit(str(seed))))) % 100
    cut = int(frac * 100)
    return (df.filter(F.col("_bucket") < cut),
            df.filter(F.col("_bucket") >= cut))

idx = StringIndexer(...); ohe = OneHotEncoder(...)
num = VectorAssembler(NUMERIC_EDIT_FEATURES, "num_raw")
sca = StandardScaler("num_raw", "num_scaled",
                     withMean=True, withStd=True)
asm = VectorAssembler(["num_scaled", "ns_ohe"]
                      + (["cmt_svd"] if use_text else []),
                      "features")
Pipeline(stages=[idx, ohe, num, sca, asm, model])
```

src/p05_clustering.py

```
# bot flag is NOT an input -- external validation only
asm = VectorAssembler(EDITOR_FEATURES, "raw")
sca = StandardScaler("raw", "features",
                     withMean=True, withStd=True)

for algo in (KMeans, BisectingKMeans,
             GaussianMixture):
    for k in [2,3,4,5,6,7,8]:
        model = algo(k=k).fit(data)
        pred  = model.transform(data)
        # k clusters requested != k NON-EMPTY
        if pred.select("cluster").distinct()
               .count() < 2:
            continue          # degenerate fit
        sil = ClusteringEvaluator().evaluate(pred)
```

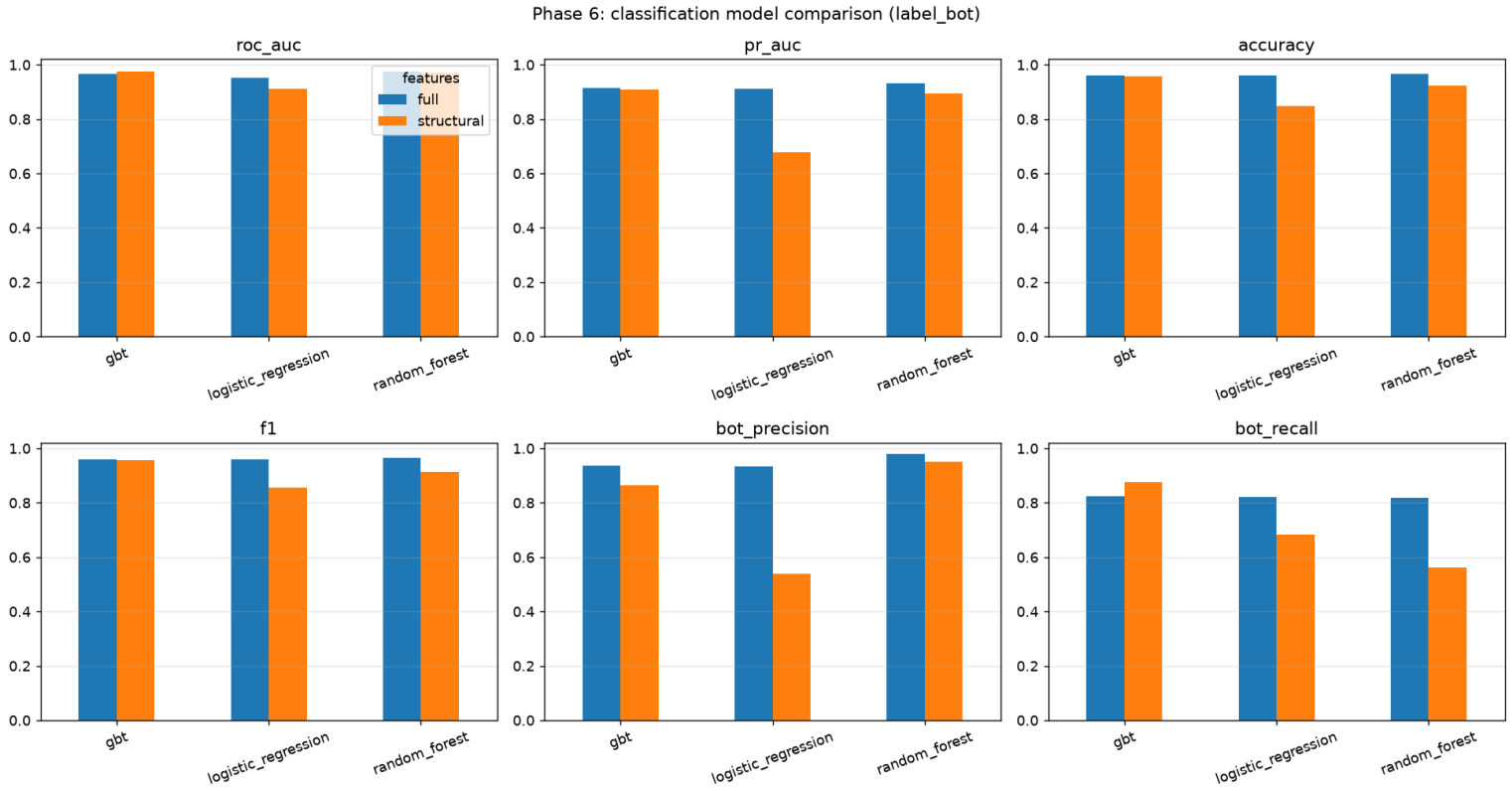
Classification: LogisticRegression, RandomForest, GBT. Clustering: KMeans, BisectingKMeans, GaussianMixture over a k-sweep, selected by silhouette.

Phase 6: Classification Results

Model	Features	ROC-AUC	PR-AUC	Accuracy	F1 (wtd)	Bot precision	Bot recall	Bot F1
Logistic Regression	structural	0.9114	0.6792	0.8489	0.8558	0.5375	0.6840	0.6020
Random Forest	structural	0.9713	0.8948	0.9219	0.9133	0.9522	0.5605	0.7057
GBT	structural	0.9756	0.9098	0.9564	0.9565	0.8648	0.8760	0.8703
Logistic Regression	full	0.9512	0.9122	0.9605	0.9594	0.9348	0.8207	0.8741
Random Forest ★	full	0.9759	0.9326	0.9667	0.9655	0.9806	0.8171	0.8914
GBT	full	0.9655	0.9144	0.9609	0.9599	0.9356	0.8229	0.8756

Best: random_forest / full at ROC-AUC 0.9759 — on a test set of accounts never seen in training.

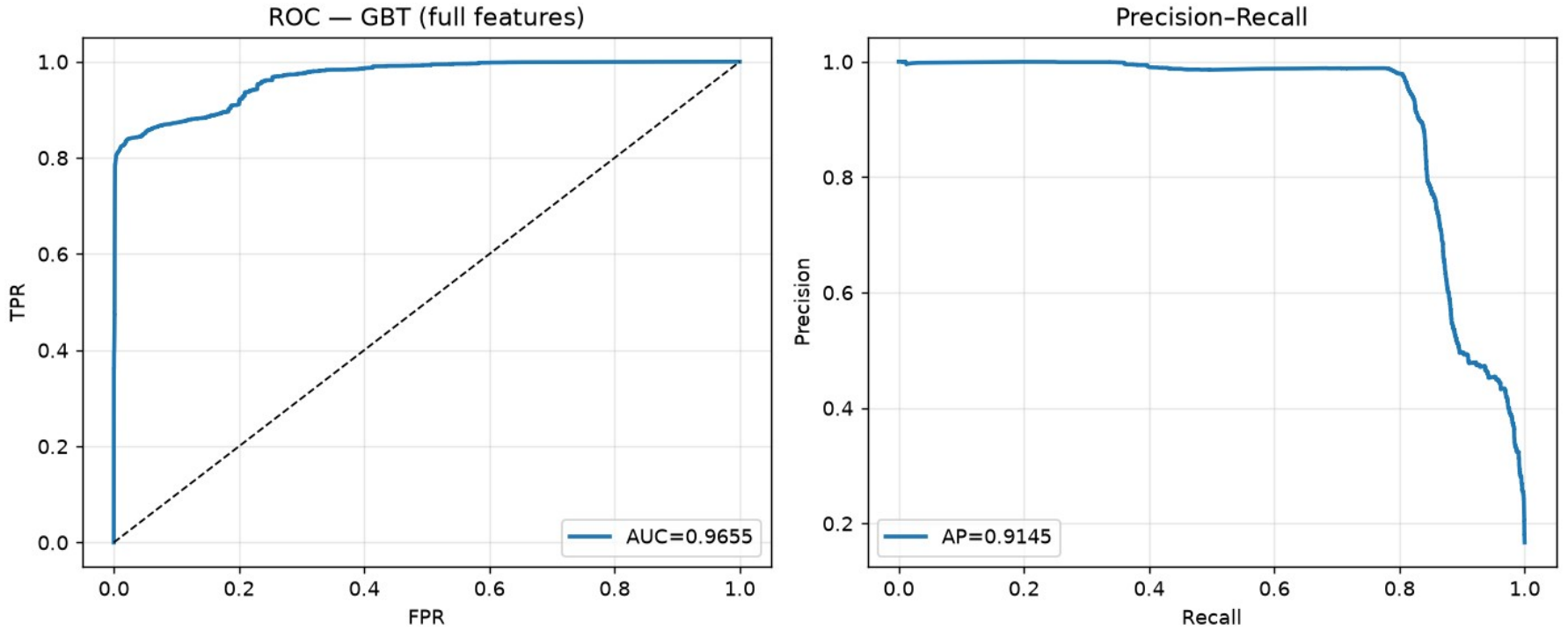
Phase 6: Structural vs Full Feature Sets



Structural behaviour alone already separates the classes. Comment text rescues the linear model but barely moves the tree ensembles -- and does not uniformly help them.

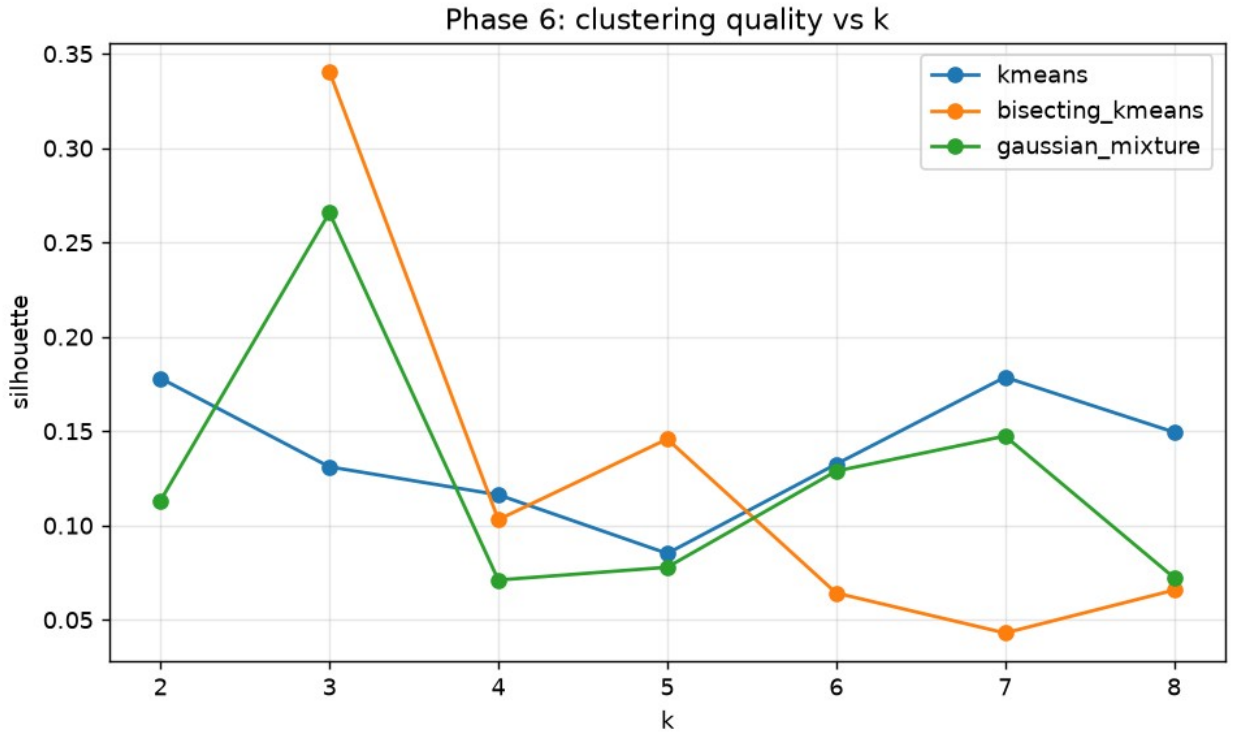
AMRITA VISHWA VIDYAPEETHAM

Phase 6: ROC and Precision–Recall Curves



Under class imbalance the precision–recall curve is the more honest view: it ignores the large true-negative mass that flatters accuracy.

Phase 6: Clustering Quality and Model Selection



Editors clustered: **108,954** of 780,867 (minimum 5 edits — single-edit accounts dominate the tail).

Best: **bisecting_kmeans_k3**, which returned **2 non-empty** clusters from 3 requested.

Silhouette: **0.3406**

Silhouette this low says the clusters are **not** well separated. Editor behaviour is a continuum, not a set of tidy groups — so we report the partition as a descriptive summary, not as discovered natural classes.

A fit can collapse: requesting k clusters does not guarantee k non-empty ones. k=2 degenerated to a single cluster and was detected and skipped rather than aborting the sweep.

AMRITA VISHWA VIDYAPEETHAM

Phase 6: Editor Archetypes Discovered

Cluster	Editors	Mean edits	Bot share	Gap CV	Night ratio	Minor ratio	Revert ratio	Edits/active day	Bot enrichment
0	82,951	12	0.001	1.73	0.20	0.03	0.396	7.1	0.19x
1	26,003	344	0.013	3.66	0.20	0.21	0.166	10.5	3.58x

A negative result, reported as one. Only 0.37% of *accounts* are bots — they are few accounts making very many edits — so cluster **purity (0.9963) is exactly the majority-class baseline (0.9963)**, a lift of +0.00000. Purity therefore carries no information here, and quoting it as validation would be misleading.

The informative measure is **enrichment**: bots are **3.6× over-represented** in the high-activity cluster. Real, but modest. **Hypothesis 3 is not supported** — the clusters separate *activity level*, not automation.

The clustering describes how much editors do, not whether they are machines. Automation is recoverable from supervised behaviour (ROC-AUC 0.976) but is not the dominant axis of unsupervised variation.

Phase 7: Scalability Discussion

Spark is not used because the file is large — it is used because the workload is a **wide shuffle**, not a scan.

Stage	Parallelisation mechanism
Raw TSV read	bz2 IS splittable in Hadoop, so the 951 MB file is read by 12 concurrent tasks (gzip would be one task)
Column projection	Parquet keeps 31 of 78 columns, so later phases read roughly a third of the bytes
Feature engineering	row-local withColumn — embarrassingly parallel, no shuffle
Editor aggregation	groupBy + window lag → sort-merge shuffle across 48 partitions; this is the expensive step
SVD	distributed RowMatrix; only V (2048 × 24) is collected and broadcast
Model training	MLlib broadcasts the model and reduces histograms per partition

Actor skew is the dominant risk at scale. Edit counts per account follow a heavy power law — a few bots have millions of revisions while the median account has one. A naive groupBy sends one enormous partition to a single task, which straggles or OOMs while other cores idle. Mitigations: adaptive skew-join (enabled), salting the heaviest keys, and bounded-memory sketches instead of exact percentiles.

Other limits: small-files problem on write (we repartition before writing), broadcast ceiling on V if TF-IDF buckets grow, and stratified sampling must happen AFTER the actor split or the disjointness guarantee breaks.

AMRITA VISHWA VIDYAPEETHAM

Results and Observations

- Automated edits are **23.7%** of all 10,987,077 edits, and **14.1%** of edits are later reverted.
- Best classifier: **random_forest / full** at ROC-AUC **0.9759**, PR-AUC **0.9326**, bot-F1 **0.8914** — on accounts never seen in training.
- **Hypothesis 2 held:** structural behaviour alone is already sufficient — GBT reaches ROC-AUC 0.9756 with no comment vocabulary at all, and adding the text block slightly reduces it (0.9655). Automation is legible from *behaviour*, not from what the edit says.
- Text matters only to the **linear** model (0.9114 → 0.9512, PR-AUC 0.6792 → 0.9122). The tree ensembles already capture the non-linear interactions that the SVD dimensions merely make visible to a linear boundary — so the ablation measures model capacity as much as feature value.
- Clustering: **bisecting_kmeans_k3** at silhouette 0.3406 over 108,954 active editors — a weak separation. **Hypothesis 3 was not supported:** the partition tracks activity level, not automation. Bots are only 3.6× enriched in the busiest cluster, and purity equals its own baseline exactly.
- Accuracy is the **least** informative metric here: with a 24/76 split, a model can look strong on accuracy while ranking poorly, which is why PR-AUC and the per-class bot-F1 are reported alongside it.
- Cleaning cost only 0.33% of rows after recovering deleted-page revisions from the *_historical columns.

Challenges Faced

1. The published schema is wrong. The docs list 76 fields; the snapshot ships 78. Trusting the doc would misalign every column, so we verified all 78 positions empirically by profiling fill-rate and value shape per index, anchored on event_entity, page_id and revision_text_sha1.

2. Raw quotes in comments silently shift columns. event_comment contains characters like "Bad words"; with default CSV quoting Spark absorbs tab delimiters inside the apparently-quoted region. Fixed with option("quote", "").

3. Deleted-page bias. 14.8% of revisions have NULL page_namespace because the page was deleted. Dropping them would have discarded 15% of the data — and deleted pages concentrate vandalism, so it would have biased label_reverted, not merely shrunk the dataset.

4. NULL booleans became NaN. NULL == "true" is NULL in SQL, so nulls survived the cast, became NaN in VectorAssembler and propagated through StandardScaler — failing deep inside a model fit with no hint of the guilty column. Fixed by coalescing, plus a null guard that fails loudly and names the column.

5. pandas_udf cannot accept VectorUDT input. The vectorised SVD projection received str and crashed. Reverted to a plain UDF with a single numpy gather+dot per row; densifying 2048 columns to vectorise would cost more than it saves.

6. A 21.5-minute SVD fit was running over 15% of 11 M rows. The right singular vectors converge with far fewer rows, so we fit on 4% and still project all rows.

7. Degenerate clustering. Requesting k clusters does not guarantee k non-empty ones, and ClusteringEvaluator then aborts the whole sweep.

8. Phase 0 decompressed the 951 MB bz2 four times, once per action. One grouped aggregate now yields the row total, entity breakdown and wiki sanity check together: four passes became two.

Conclusion and Future Scope

Conclusion

We built a complete Spark MLlib pipeline over 10,987,077 real Wikimedia edits: Parquet ingestion, DataFrame cleaning with an enforced null contract, 35 engineered behavioural features plus a 24-dimension truncated SVD of comment text, three classifiers and three clustering algorithms, all evaluated under an actor-disjoint split. Automation is predictable from behaviour alone (ROC-AUC 0.9759). Unsupervised clustering of the same editors, by contrast, separates activity level rather than automation — bots are only 3.6x enriched in the busiest cluster. We report that as a negative result: automation is learnable with supervision but is not the dominant axis of unsupervised behavioural variation.

Known limitations (stated rather than hidden)

The label covers only **flagged** accounts, so unapproved bots are false negatives by construction and recall is measured against an incomplete positive class.

Semi-automated human tooling (AWB, Huggle) genuinely sits between the classes; no clean boundary exists there.

The all-time snapshot spans 2001–2026, averaging over several distinct editing regimes.

Future scope: scale to enwiki monthly shards on a real cluster; score the live EventStreams feed with the trained model; model label_reverted as a second task; treat era as a stratum.

References

[1] Wikimedia Foundation. MediaWiki History Dumps. https://dumps.wikimedia.org/other/mediawiki_history/ (CC0).

[2] Wikimedia Foundation. EventStreams. https://stream.wikimedia.org/v2/stream/recentchange

[3] A. Halfaker and D. Taraborelli. "ORES: Scaling up machine learning-driven quality control." Wikimedia, 2015.

[4] M. Potthast, B. Stein and R. Gerling. "Automatic Vandalism Detection in Wikipedia." ECIR, 2008.

[5] B. T. Adler et al. "Wikipedia Vandalism Detection: Combining Natural Language, Metadata and Reputation Features." CICLing, 2011.

[6] C. A. Davis et al. "BotOrNot: A System to Evaluate Social Bots." WWW (demo), 2016.

[7] Z. Chu et al. "Detecting Automation of Twitter Accounts: Are You a Human, Bot, or Cyborg?" IEEE TDSC, 2012.

[8] H. T. Welser et al. "Finding Social Roles in Wikipedia." iConference, 2011.

[9] S. Kumar, F. Spezzano and V. S. Subrahmanian. "VEWS: A Wikipedia Vandal Early Warning System." KDD, 2015.

[10] M. Zaharia et al. "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing." NSDI, 2012.

[11] X. Meng et al. "MLlib: Machine Learning in Apache Spark." JMLR 17(34), 2016.

[12] Apache Spark documentation, version 3.5.3. https://spark.apache.org/docs/3.5.3/ml-guide.html

THANK YOU